Alright — I'll convert your understanding into a **clean, production-grade engineering document** that you can directly use in your project (README / design doc / internal wiki).

📄 PIM-AI — Databricks Integration Design

(Product Description Generator — Content Agent)

---

1. 🎯 Purpose

This document defines the role of **Databricks** in the PIM-AI architecture, specifically for the **Product Description Generator Agent**.

It clarifies:

- Where Databricks fits in the system
- What problems it solves
- What it does *and does NOT do*
- How it integrates with existing architecture (FastAPI + LangGraph + MCP)

---

2. 🧠 Architectural Context

PIM-AI follows the **Custonomy multi-agent architecture principle**:

> "Process once, decide many times"

And:

> "Agents consume feature tables, not raw data"

🧩 Logical Flow

```
Raw Product Data → Databricks → Feature Tables → Signals → Agent (LangGraph) → LLM (Claude)
```

---

3. 🏗️ System Placement

Databricks belongs to:

✔️ Layer 2 — Feature Computation Layer

✔️ Layer 3 — Signal Layer

From Custonomy architecture:

- Feature Layer = reusable computed data
- Signal Layer = decision triggers

---

4. 🚫 What Databricks is NOT

Databricks is **NOT responsible for:**

- Agent orchestration (LangGraph)
- API layer (FastAPI)
- Tool execution (MCP / FastMCP)
- LLM inference (Claude API)

👉 These remain inside your existing system.

---

5. ✅ Core Responsibilities of Databricks

5.1 Feature Engineering (PRIMARY ROLE)

Databricks computes reusable **feature tables** used by the Content Agent.

Example Features for Description Generation

Feature Type	Example
Category Intelligence	Top keywords, attribute importance
SEO Signals	High-ranking keywords per category
Content Patterns	Best-performing descriptions
Data Quality	Missing fields, completeness score
Product Context	Attribute distributions

❌ Without Databricks

```
LLM ← raw product data (expensive, inconsistent)
```

✅ With Databricks

```
LLM ← product data + structured features (efficient, consistent)
```

5.2 Signal Generation (Trigger Layer)

Databricks computes **decision signals**:

- `missing_description_flag`
- `low_quality_content_flag`
- `seo_gap_flag`

👉 These signals control **agent invocation**

Instead of:

❌ Agent scanning entire catalog

You get:

✔️ Event-driven execution

5.3 Data Pipelines

Databricks handles:

- Batch processing (large catalog)
- Incremental updates
- External data joins (SEO sources, competitors)
- Feature refresh jobs

5.4 Model Training (OPTIONAL)

Databricks may be used for:

- Fine-tuning content generation models

- Training:
    - SEO scoring models
    - Content quality scoring (E-14)

⚠️ Not required in Phase 1:

- Claude API already handles generation

5.5 Evaluation Pipelines

Supports:

- Large-scale evaluation datasets
- A/B testing descriptions
- Measuring:
    - SEO performance
    - readability
    - conversion impact

---

6. 🔄 Integration with Content Agent

6.1 Data Flow

Step 1 — Databricks (Offline Processing)

- Compute features:
    - keywords
    - attribute importance
    - SEO signals
    - content patterns

Step 2 — Storage Layer

- Store in:

- PostgreSQL (feature tables)
- Delta tables (Databricks)

Step 3 — Agent Execution (Online)

Content Agent workflow:

```
1. Fetch product data
2. Fetch precomputed features
3. Call generate_description tool
4. Use Claude API
5. Store output
```

6.2 Workflow Mapping

From your task doc:

> Product onboarding workflow:
> classify → tag → generate description → sequence images

👉 Databricks enriches the **generate_description step**

---

7. 🧠 Example: Product Description Generation Flow

Input

- Product attributes
- Category
- Images
- Feature table data (Databricks)

Processing

```
generate_description(
    product,
    category_keywords,
```

```
    seo_signals,
    attribute_importance,
    content_patterns
)
```

Output

- SEO-optimized description
- Title
- Metadata

Result (from feature doc)

- Consistent tone
- Faster generation
- SEO improvement
- Reduced manual effort

---

8. 🏗️ Reference Architecture

```
┌──────────────────────┐
│     Databricks       │
├──────────────────────┤
│ Feature Engineering  │
│ Signal Generation    │
│ Batch Pipelines      │
└──────────────────────┘
          ↓
┌──────────────────────┐
│ PostgreSQL / APIs    │
│ Feature Tables       │
└──────────────────────┘
          ↓
┌──────────────────────┐
│ Content Agent        │
│ (LangGraph)          │
└──────────────────────┘
          ↓
Claude API (LLM)
```

---

9. ⚖️ When to Use Databricks

✅ Use Databricks if:

- Large catalog scale (100K+ products)
- Complex feature computation
- Need reusable feature store
- Need ML pipelines

❌ Avoid Databricks if:

- Small dataset
- Simple pipelines
- Early-stage MVP

👉 Your ADR prioritizes **low infra complexity**
→ Databricks is **optional initially**

---

10. 🧩 Key Design Principles

---

1. Process Once, Reuse Everywhere

- Databricks computes features once
- All agents reuse them

---

2. No Raw Data Access by Agents

- Agents consume APIs / feature tables only

---

3. Event-Driven Execution

- Signals trigger agents
- No full catalog scans

---

4. Separate Offline vs Online Compute

Layer	Responsibility
Databricks	Heavy compute, feature generation
Agent	Decision making + LLM

---

11. 🚀 Future Evolution

Phase 1 (Current)

- Claude API
- Minimal features
- No Databricks required

Phase 2

- Introduce feature tables
- Basic Databricks pipelines

Phase 3

- ML models (SEO scoring, quality scoring)
- Fine-tuning

Phase 4

- Full feature store
- Cross-agent shared intelligence

---

12. 🧠 Final Summary

👉 Databricks is a **data + feature + ML layer**, NOT an agent layer

👉 It enables:

- smarter LLM inputs
- scalable processing
- event-driven agents

👉 It integrates like this:

```
Databricks → Features → Agents → LLM → Actions
```

🔥 If you want next level

I can extend this into:

- Exact **Databricks table schema**
- Feature definitions (columns + logic)
- Job design (batch + streaming)
- Integration APIs with FastAPI

Just say the word.